

CSE_2323

Combinational Logic

Digital Logic Design

Course code: CSE-2323

Credit Hour: 3

Md. Mujibur Rahman Maruf

ASSISTANT LECTURER,

Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

Combinational Circuits

- **Logic circuits** for digital systems may be **combinational** or **sequential**.
- A combinational circuit consists of input variables, logic gates, and output variables.

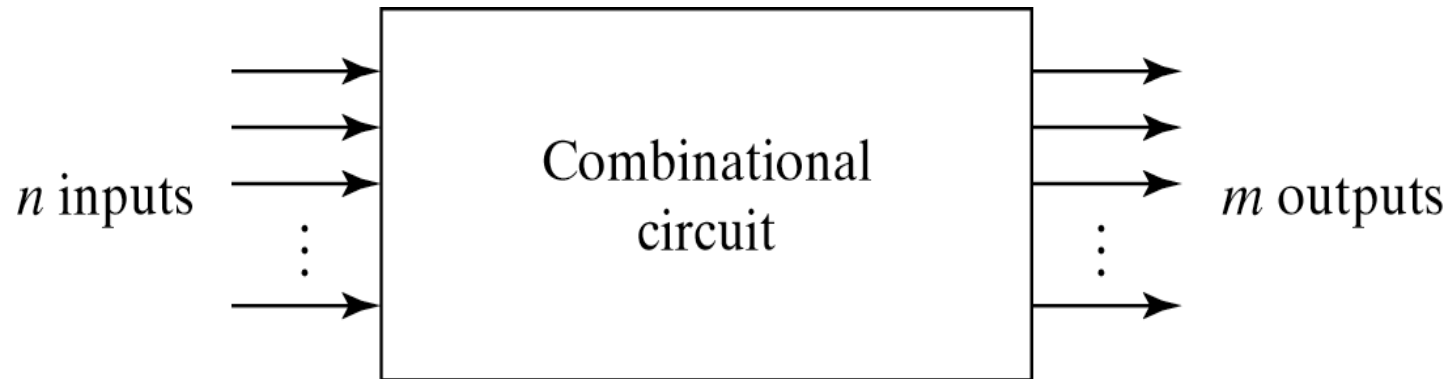


Fig. 4-1 Block Diagram of Combinational Circuit

Examples of Combinational Circuits

- *Addition:*
 - *Half Adder (HA).*
 - *Full Adder (FA).*
 - *BCD(Decimal) Adder.*
- *Subtraction:*
 - *Half Subtractor.*
 - *Full Subtractor.*
- *Multiplication:*
 - *Binary Multipliers.*
- *Comparator:*
 - *Magnitude Comparator.*

Examples of Combinational Circuits

- *Multiplexers*
- *Demultiplexers*
- *Encoders*
- *Decoders*
- *Converters*
 - *Binary to Gray Code*
 - *Gray to Binary Code*
 - *Binary to BCD Code*

Designing Combinational Circuits

In general we have to do following steps:

- 1. Problem description*
- 2. Input/output of the circuit*
- 3. Define truth table*
- 4. Simplification for each output*
- 5. Draw the circuit*

Half Adder

- Adding two single-bit binary values, A , B produces a sum S bit and a carry out C -out bit.
- This operation is called half addition and the circuit to realize it is called a half adder.



Half Adder Truth Table

Input		Outputs	
x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Sum	X \ Y	Y'	Y
		0	1
X'	0	0	1
	1	1	0

$$S = XY' + X'Y$$

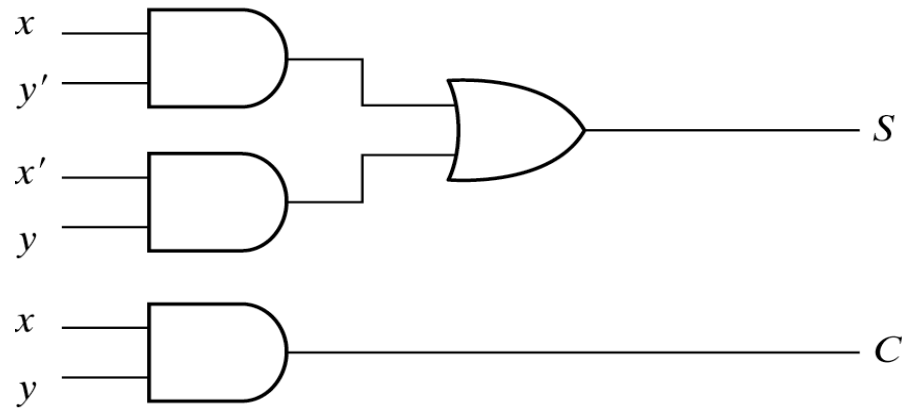
$$S = X \oplus Y$$

C	X \ Y	Y'	Y
		0	1
X'	0	0	0
	1	0	1

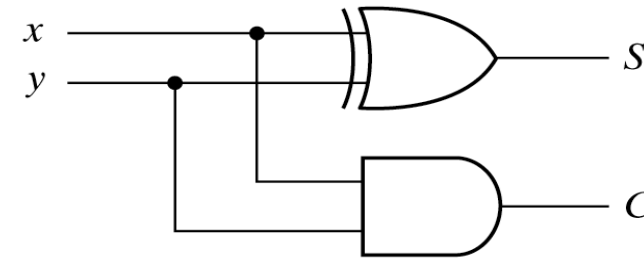
$$C = XY$$

Half Adder

$$S = XY' + X'Y \quad C = XY$$
$$S = X \oplus Y$$



(a) $S = xy' + x'y$
 $C = xy$



(b) $S = x \oplus y$
 $C = xy$

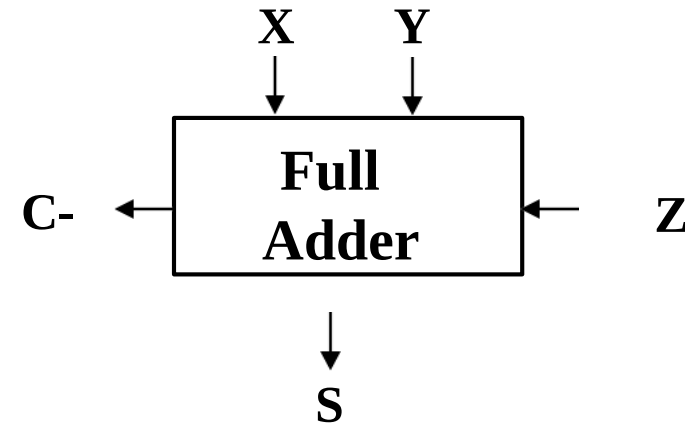
Fig. 4-5 Implementation of Half-Adder

Full Adder

- Adding two single-bit binary values, X, Y with a carry input bit C-in produces a sum bit S and a carry out C-out bit.

Full Adder Truth Table

Inputs			Outputs	
X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Think of Z as a carry in

Full Adder

- A combinational circuit that adds 3 input bits to generate a Sum bit and a Carry bit

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Sum		YZ	Y'Z'	Y'Z	YZ	YZ'
		X	00	01	11	10
x'	0	0	0	1	0	1
	1	1	0	1	0	

$$S = X'Y'Z + X'YZ' + XY'Z' + XYZ$$

Carry		YZ	Y'Z'	Y'Z	YZ	YZ'
		X	00	01	11	10
x'	0	0	0	0	1	0
	1	0	1	1	1	1

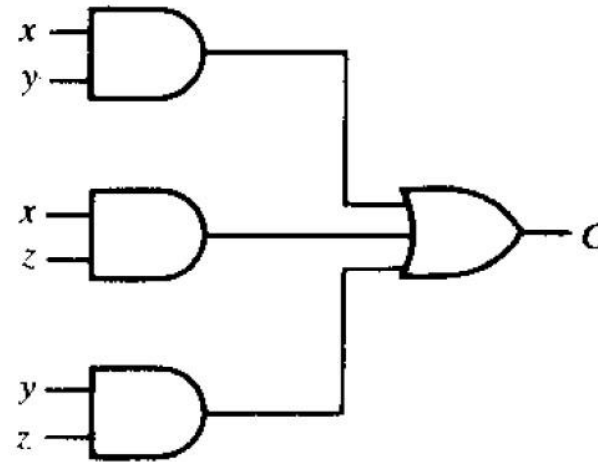
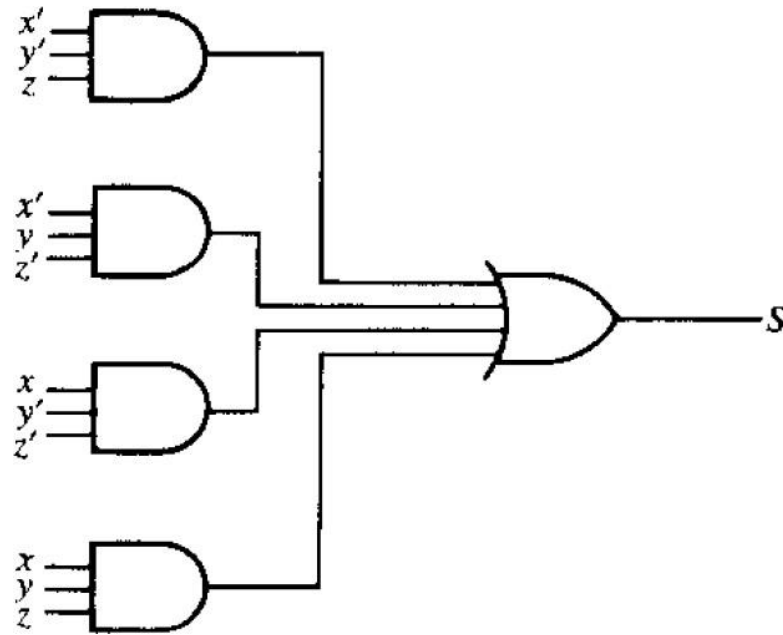
$$C = XY + XZ + YZ$$

Full Adder

Implementation of full adder in sum of products

$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$C = xy + xz + yz$$



Implementation of full adder with two half-adders and an OR gate

Manipulating the Equations:

Sum

X \ YZ	00	01	11	10
x' 0	0	1	0	1
x 1	1	0	1	0

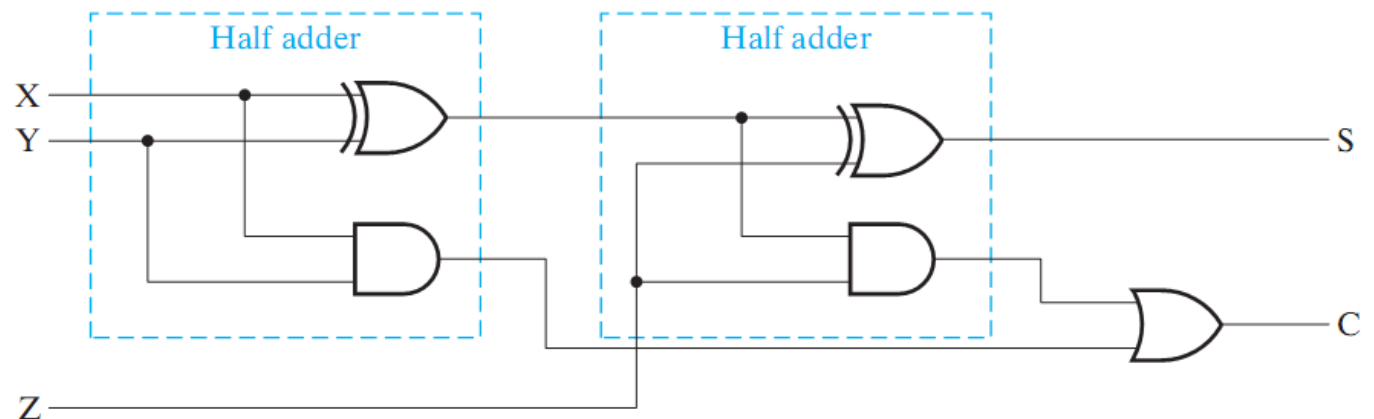
$$\begin{aligned}
 S &= x'y'z + x'yz' + xy'z' + xyz \\
 &= xy'z' + x'yz' + xyz + x'y'z \\
 &= z'(xy' + x'y) + z(xy + x'y') \\
 &= z'(x \oplus y) + z(x \oplus y)' \\
 &= z \oplus (x \oplus y)
 \end{aligned}$$

Think of
Z as a
carry in

Carry

X \ YZ	00	01	11	10
x' 0	0	0	1	0
x 1	0	1	1	1

$$\begin{aligned}
 C &= XY'Z + XYZ + X'YZ + XYZ' \\
 &= Z(XY' + X'Y) + XY(Z + Z') \\
 &= XY + Z(X \oplus Y)
 \end{aligned}$$



Binary Parallel Adder

- To add n-bit numbers:
- Use n Full-Adders in parallel
- The carries propagates as in addition by hand
- For example to add A= 1011 and B= 0011

subscript i: 3 2 1 0

Input carry: 0 1 1 C_i

Augend: 1 0 1 1 A_i

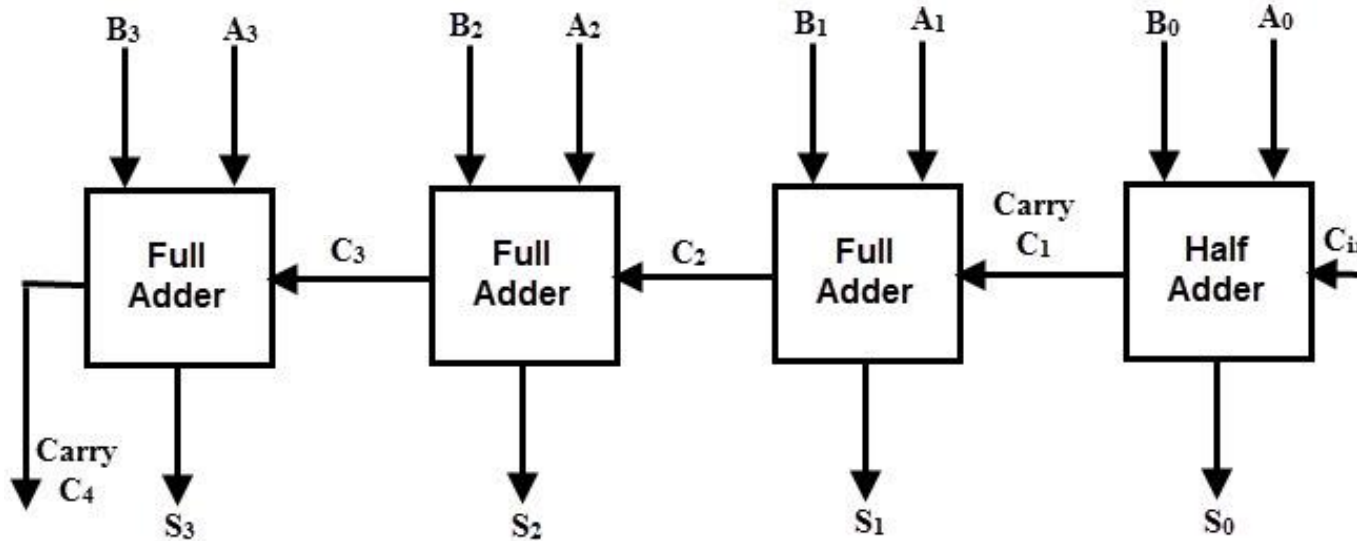
Addend: 0 0 1 1 B_i

Sum: 1 1 1 0 S_i

Output carry: 0 0 1 1

Binary Parallel Adder

- To add n-bit numbers:
- Use n Full-Adders in parallel
- The carries propagates as in addition by hand



Src: Mano's Book

This adder is called *ripple carry adder*

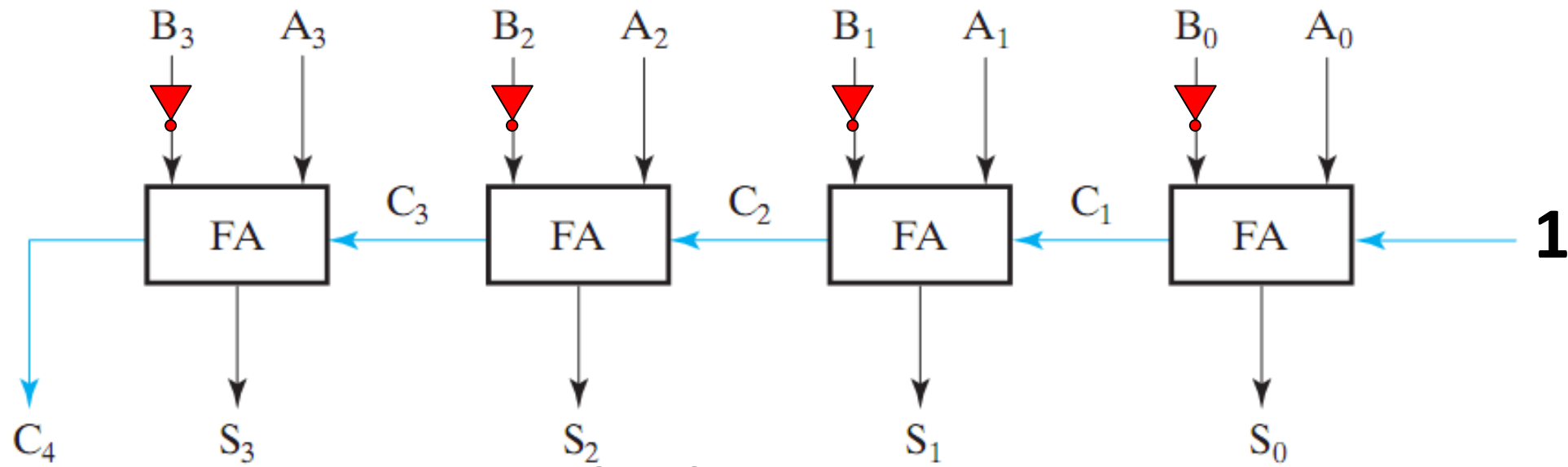
Subtraction (2's Complement)

- How to build a subtractor using 2's complement?

$$\begin{array}{r}
 \text{A} \quad 13 \qquad \qquad \qquad 1 \ 0 \ 1 \ 0 \text{ Borrow} \\
 \text{B} - \quad 6 \ 1\text{'s} \qquad \qquad 1 \ 1 \ 0 \ 1 \\
 \hline
 7 \qquad \qquad \qquad \qquad \qquad 1 \ 0 \ 0 \ 1 \\
 \hline
 \qquad \qquad \qquad \qquad \qquad 1 \ 0 \ 1 \ 1 \ 0 \\
 \qquad \qquad \qquad \qquad \qquad \qquad + 1 \\
 \qquad \qquad \qquad \qquad \qquad \hline
 \qquad \qquad \qquad \qquad \qquad 0 \ 0 \ 1 \ 1 \ 1
 \end{array}$$

Subtraction (2's Complement)

- How to build a subtractor using 2's complement?



$$S = A + (-B)$$

Half Subtractor

- Subtracting a single-bit binary value B from another A (I.e. $A-B$) produces a difference bit D and a borrow out bit $B\text{-out}$.
- This operation is called half subtraction and the circuit to realize it is called a half subtractor.

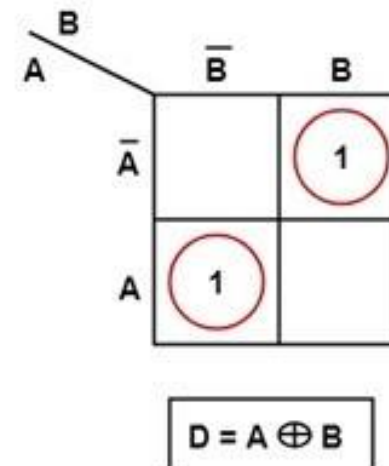
Half Adder Truth Table

Inputs		Outputs	
A	B	D	$B\text{-out}$
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

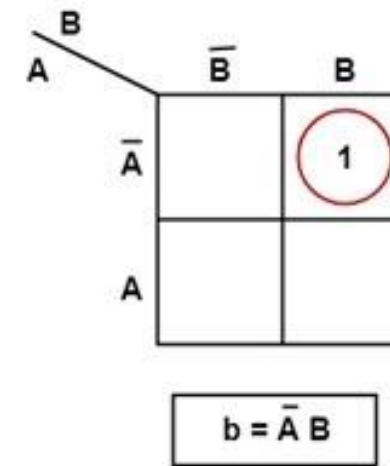
$$D = A'B + AB'$$

$$B\text{-out} = A'B$$

For D :



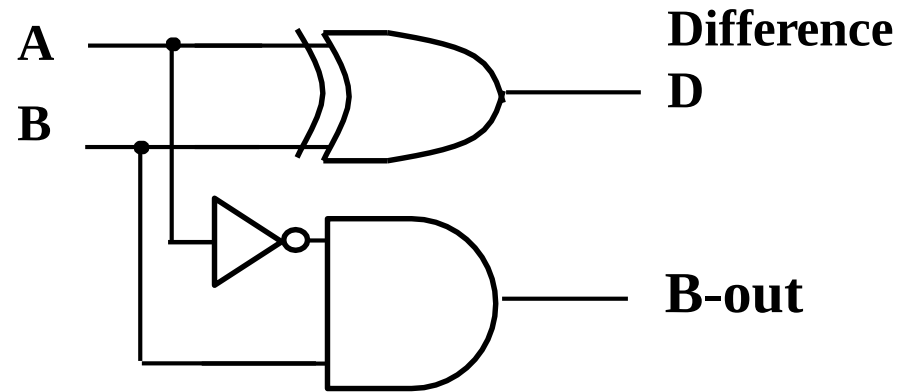
For b :



Half Subtractor

$$D = A'B + AB'$$

$$\text{B-out} = A'B$$



Full Subtractor

- Subtracting two single-bit binary values, B , B -in from a single-bit value A produces a difference bit D and a borrow out B -out bit. This is called full subtraction.

Full Subtractor Truth Table

Inputs			Outputs	
A	B	B _{in}	D	B _{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$D(A,B, B\text{-in}) = \Sigma (1,2,4,7)$$

$$B\text{-out}(A, B, B_{in}) = \Sigma (1,2,3,7)$$

Full Subtractor

Full Subtractor Truth Table

Inputs			Outputs	
A	B	B _{-in}	D	B _{-out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



$$D = A \oplus B \oplus B_{in}$$

$$D = A'B'(B_{in}) + AB'(B_{in})' + A'B(B_{in})' + AB(B_{in})$$

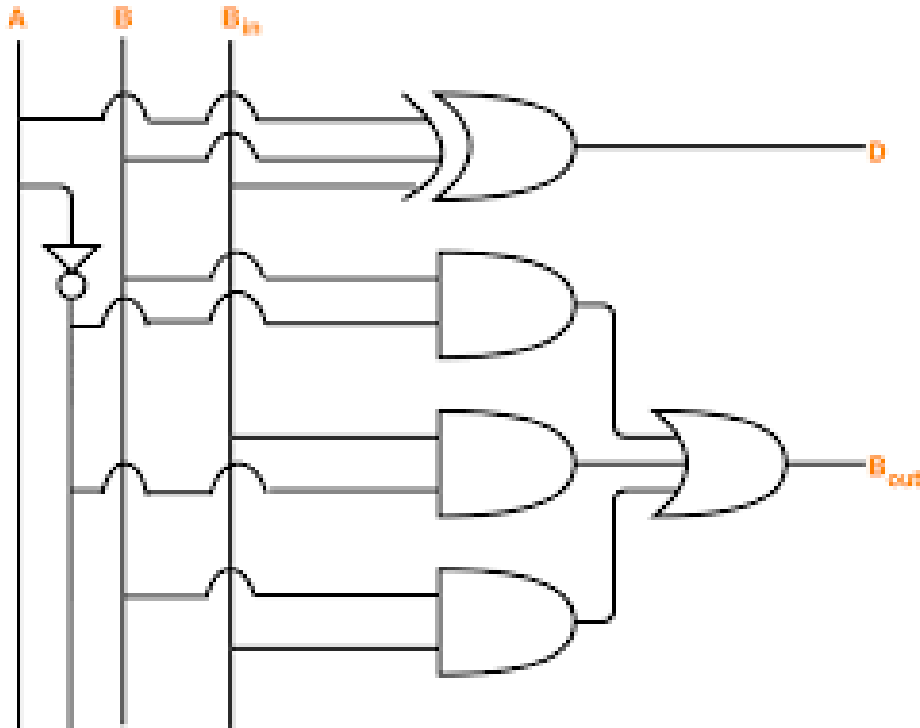
$$D = A \oplus B \oplus (B_{in})$$

$$D(A, B, B_{in}) = \Sigma (1, 2, 4, 7)$$

$$B_{out}(A, B, B_{in}) = \Sigma (1, 2, 3, 7)$$

Full Subtractor

$$B_{out} = A'B + A'(B-in) + B(B-in)$$



Full Subtractor Logic Diagram

	BB_{in}	$\bar{B}\bar{B}_{in}$	$\bar{B}B_{in}$	BB_{in}	$B\bar{B}_{in}$
$\bar{A}$		1	1	1	
A			1		

$$B_{out} = \bar{A}B + (\bar{A} + B)B_{in}$$

Full Subtractor Using half subtractor

$$D = A \oplus B \oplus (B_{in})$$

$$B\text{-out} = A'B'B_{in} + A'BB'_{in} + ABB_{in} + A'BB_{in}$$

$$B\text{-out} = B_{in}(A'B' + AB) + A'B(B'_{in} + B_{in})$$

$$B\text{-Out} = B_{in}(AB' + A'B)' + A'B$$

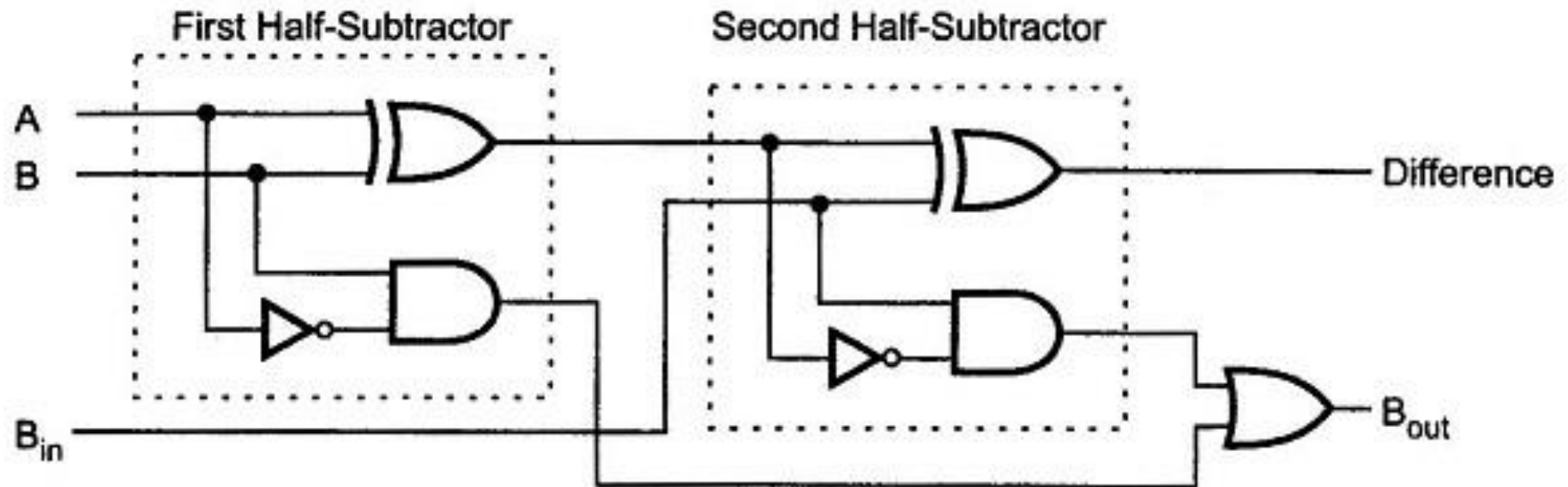
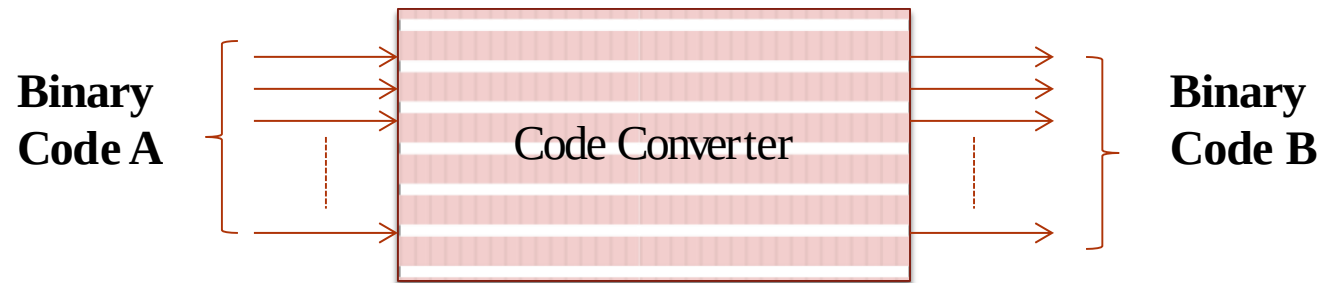


Fig. 3.24 Implementation of a full-subtractor with two half-subtractors and an OR gate

CODE Converter

A code converter is a combinational circuit that makes the two systems compatible even though each uses a different binary code.



Binary to Gray CODE Converter

truth table explaining the operation of a 4-bit binary-to-gray code converter.

Binary Code				Gray Code			
B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

Binary to Gray CODE Converter

The K-Map simplification for the gray code bit G_0

B_1B_0	00	01	11	10
B_3B_2				
00	0	1	3	2
01	4	1	5	6
11	12	1	13	14
10	8	1	9	10

Hence, the Boolean expression for the gray code bit G_0 is,

$$G_0 = \overline{B_1} B_0 + B_1 \overline{B_0} = B_0 \oplus B_1$$

The K-Map simplification for the gray code bit G_2

B_1B_0	00	01	11	10
B_3B_2				
00	0	1	3	2
01	1	1	1	1
11	12	13	15	14
10	1	1	1	1

The Boolean expression for the gray code bit G_2 will be,

$$G_2 = \overline{B_3} B_2 + B_3 \overline{B_2} = B_2 \oplus B_3$$

The K-Map simplification for the gray code bit G_1

B_1B_0	00	01	11	10
B_3B_2				
00	0	1	3	2
01	4	1	5	6
11	12	1	13	14
10	8	1	9	10

Thus, the Boolean expression for the gray code bit G_1 is,

$$G_1 = \overline{B_2} B_1 + B_2 \overline{B_1} = B_1 \oplus B_2$$

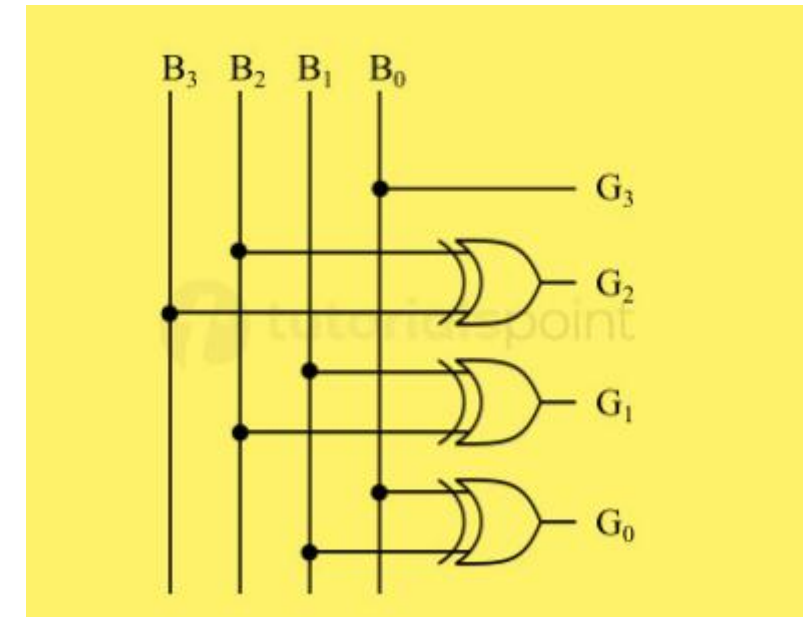
The K-Map simplification for the gray code bit G_3

B_1B_0	00	01	11	10
B_3B_2				
00	0	1	3	2
01	4	5	7	6
11	1	1	1	1
10	1	1	1	1

Hence, the Boolean expression for the gray code bit G_3 is,

$$G_3 = B_3$$

the logic circuit diagram of a 4-bit binary code to gray code converter –



Problem-1

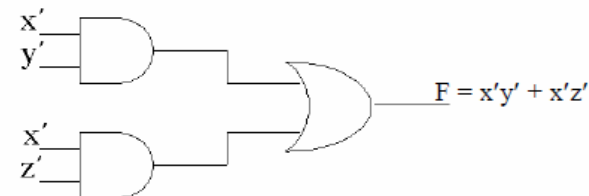
A committee of three individuals decide issues for an organization. Each individual votes either yes or no for each proposal that arises. A proposal is passed if it receives at least two yes votes. Design a circuit that determines whether a proposal passes

Problem-2

Design a combinational circuit with three inputs and one output. The output is 1 when the binary value of the inputs is less than 3. The output is 0 otherwise

x	y	z	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

	$\longleftrightarrow y$			
x\yz	00	01	11	10
0	1	1		1
1				



Problem-3

Design a combinational circuit with three inputs, x , y , and z , and three outputs, A , B , and C . When the binary input is 0, 1, 2, or 3, the binary output is one greater than the input. When the binary input is 4, 5, 6, or 7, the binary output is less than the input.

x	y	z	A	B	C
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	0	1	1
1	0	1	1	0	0
1	1	0	1	0	1
1	1	1	1	1	0

		y			
	$x \backslash yz$	00	01	11	10
x	0			1	
	1		1	1	1

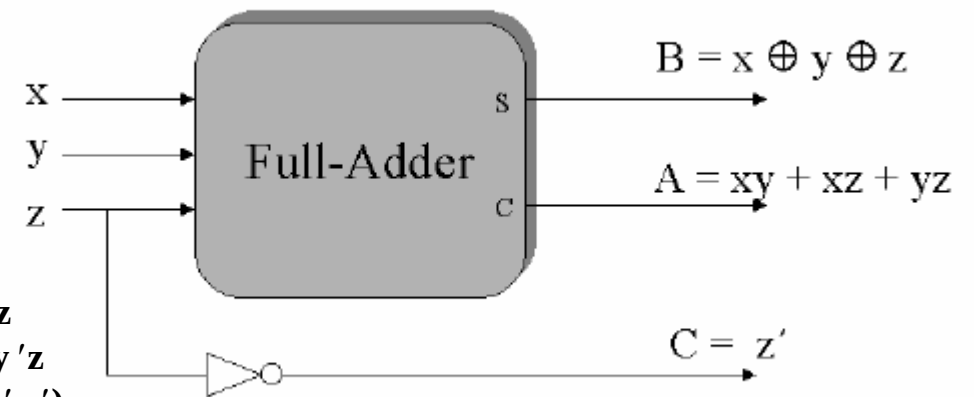
$$A = xy + xz + yz$$

		y			
	$x \backslash yz$	00	01	11	10
x	0		1		1
	1	1		1	

$$B = x \oplus y \oplus z$$

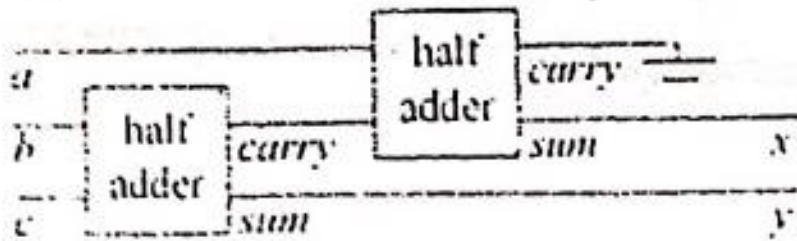
$$C = z'$$

$$\begin{aligned}
 B &= x'y'z + x'yz' + xy'z' + xyz \\
 &= xy'z' + x'yz' + xyz + x'y'z \\
 &= z'(xy' + x'y) + z(xy + x'y') \\
 &= z'(x \oplus y) + z(x \oplus y)' \\
 &= z \oplus (x \oplus y)
 \end{aligned}$$



Problem-4

Fill in the truth table at right for the following circuit and derive the Boolean function for x and y .

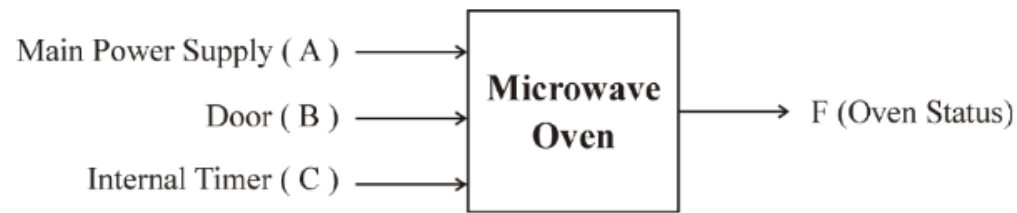


a	b	c	x	y
0	1	1		
1	0	1		
1	1	0		
1	1	1		

Problem-5

A microwave oven requires that the main power supply be on, the door be latched, and timer set other than zero before the oven will function. Draw a logic diagram for the operation of the oven.

Solution:



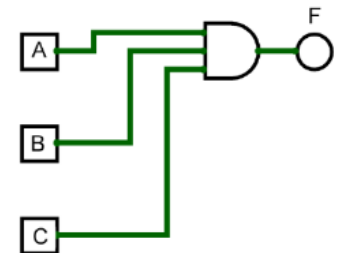
The inputs and outputs of the logic circuit are defined as;

Variable	Type	Brief Description
A	Input	If A = 1, Main power supply is ON else it is OFF.
B	Input	If B = 1, Door is latched else not
C	Input	If C = 1, timer is set other than zero, else not
F	Output	If F = 1, Microwave oven is functioning else not

Input Output

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

The circuit diagram is shown below;



Problem-6

1. Construct a truth table for a circuit that will accept 3 bit numbers and generate the 2's complement of these numbers.

Problem-7

Design a combinational circuit that converts 3-bit binary number to Gray code

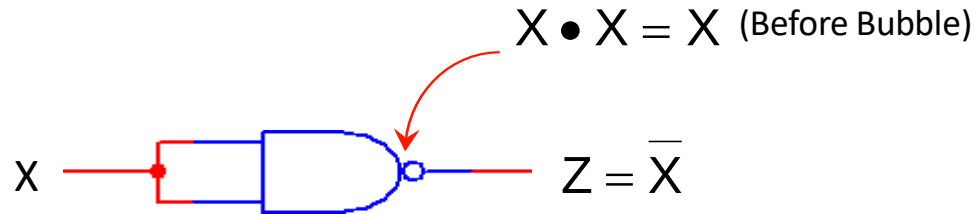
A B C	X Y Z
0 0 0	0 0 0
0 0 1	0 0 1
0 1 0	0 1 1
0 1 1	0 1 0
1 0 0	1 1 0
1 0 1	1 1 1
1 1 0	1 0 1
1 1 1	1 0 0

NAND and NOR Implementations

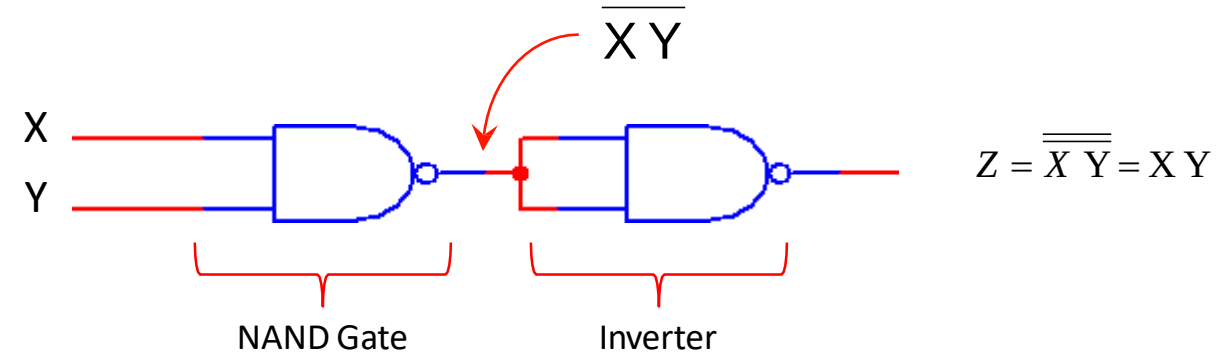
- *NAND & NOR gates are universal gates.*
- *Digital circuit are frequently constructed with NAND or NOR gates rather than AND and OR gates.*
- *NAND and NOR gates are easier to fabricate with electronic components and are the basic gates used in all IC digital logic families.(covering AND,OR,NOT)*

NAND Gate Equivalent to AOI Gates

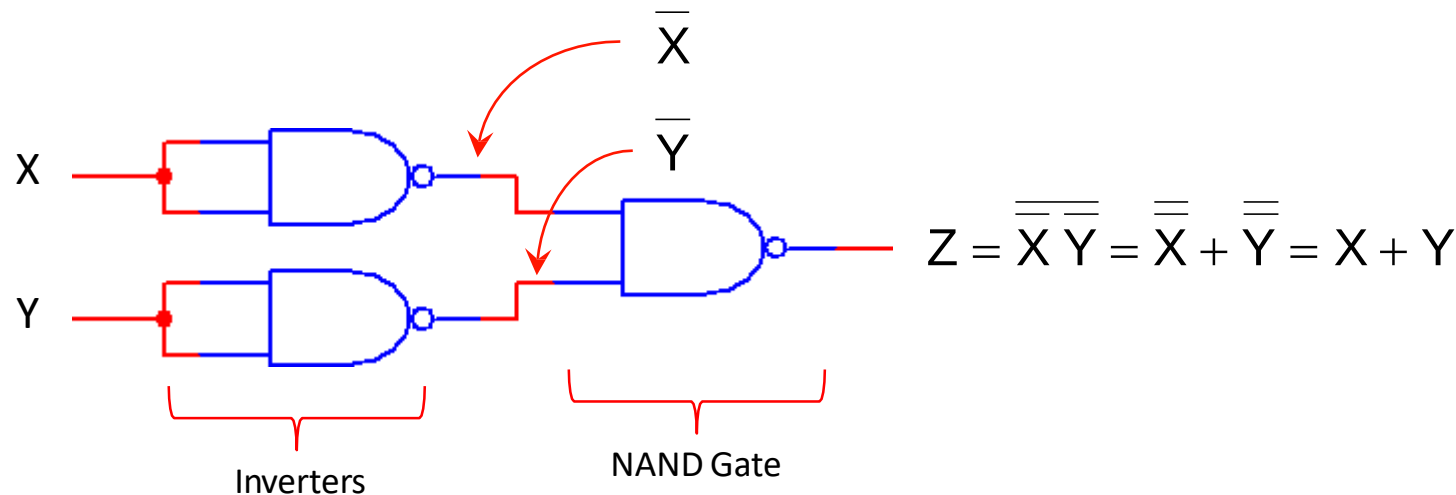
NAND Gate as an Inverter Gate



NAND Gate as an And Gate

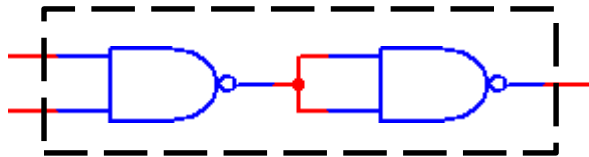
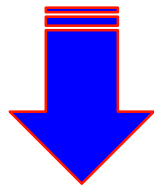


NAND Gate as an Or Gate

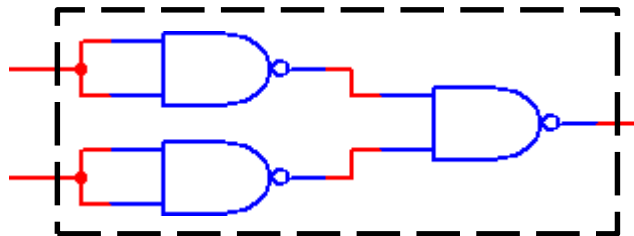
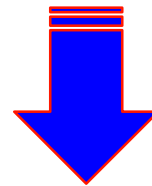


NAND Gate Equivalent to AOI Gates

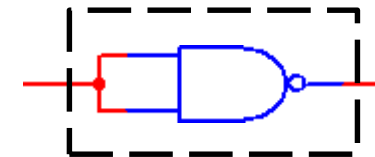
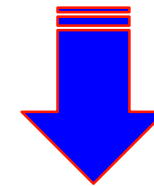
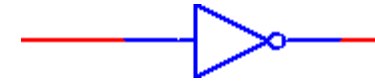
AND



OR



INVERTER



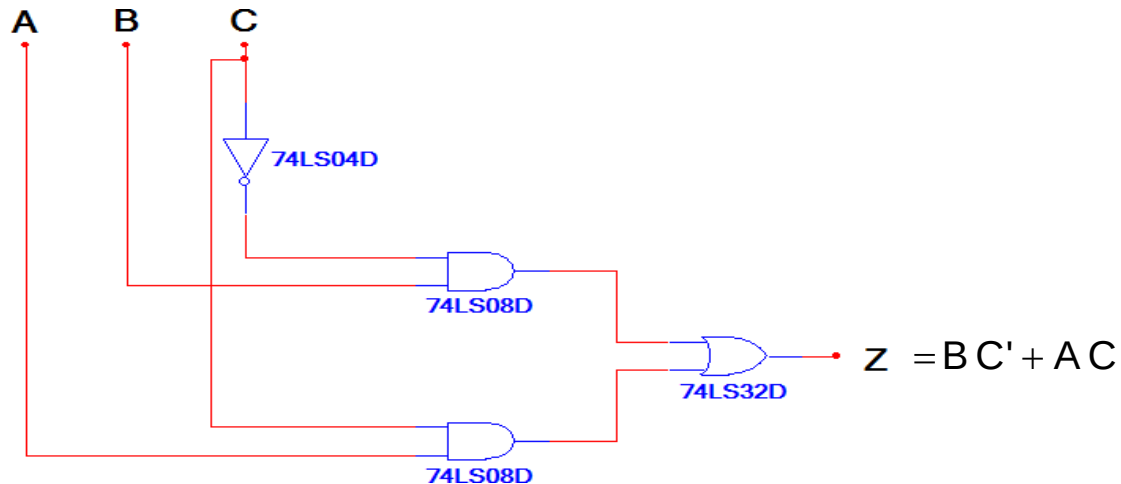
Process for NAND Implementation

1. If starting from a logic expression, implement the design with AOI logic.
2. In the AOI implementation, identify and replace every AND, OR, and INVERTER gate with its NAND equivalent.
3. Redraw the circuit.
4. Identify and eliminate any double inversions (i.e., back-to-back inverters).
5. Redraw the final circuit.

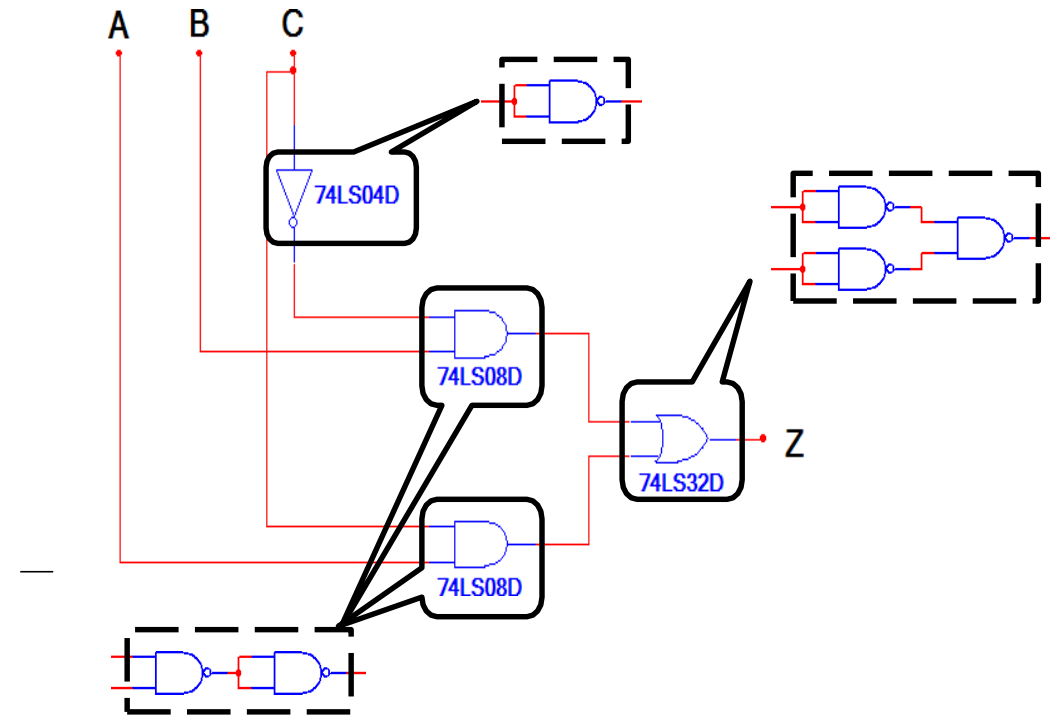
NAND Implementation

Example:

Design a NAND Logic Circuit that is equivalent to the AOI circuit shown below.

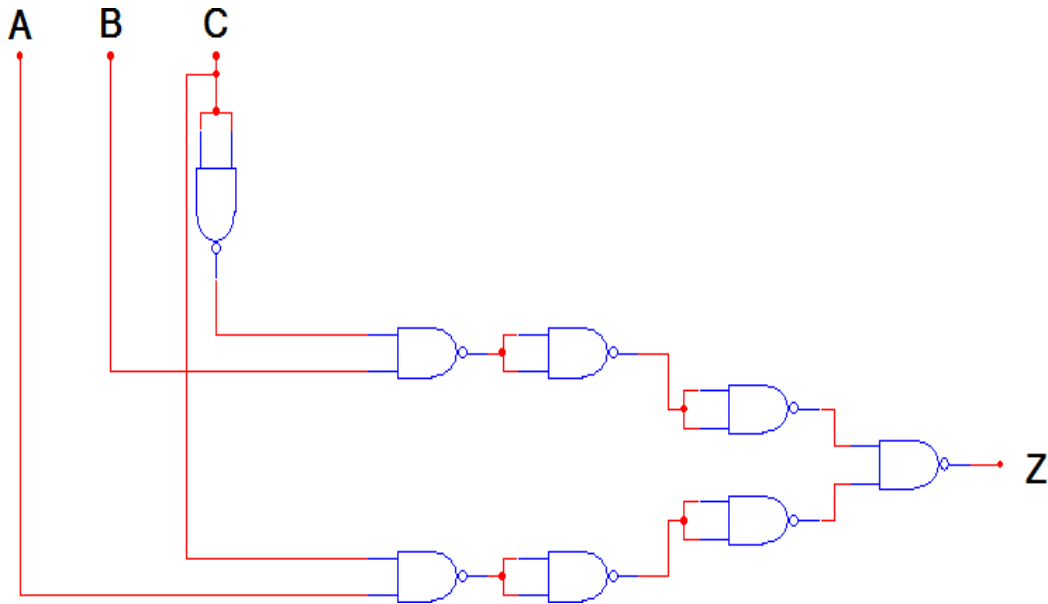


Step:2 Identify and replace every AND, OR, and INVERTER gate with its NAND equivalent.

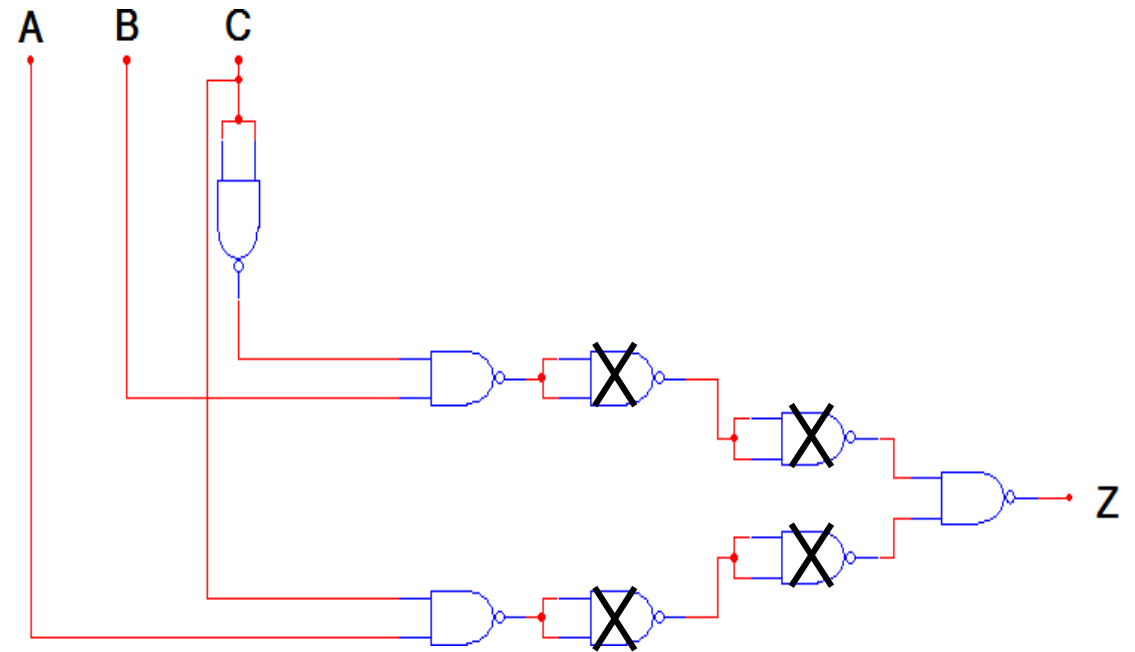


NAND Implementation

Step 3: Redraw the circuit.

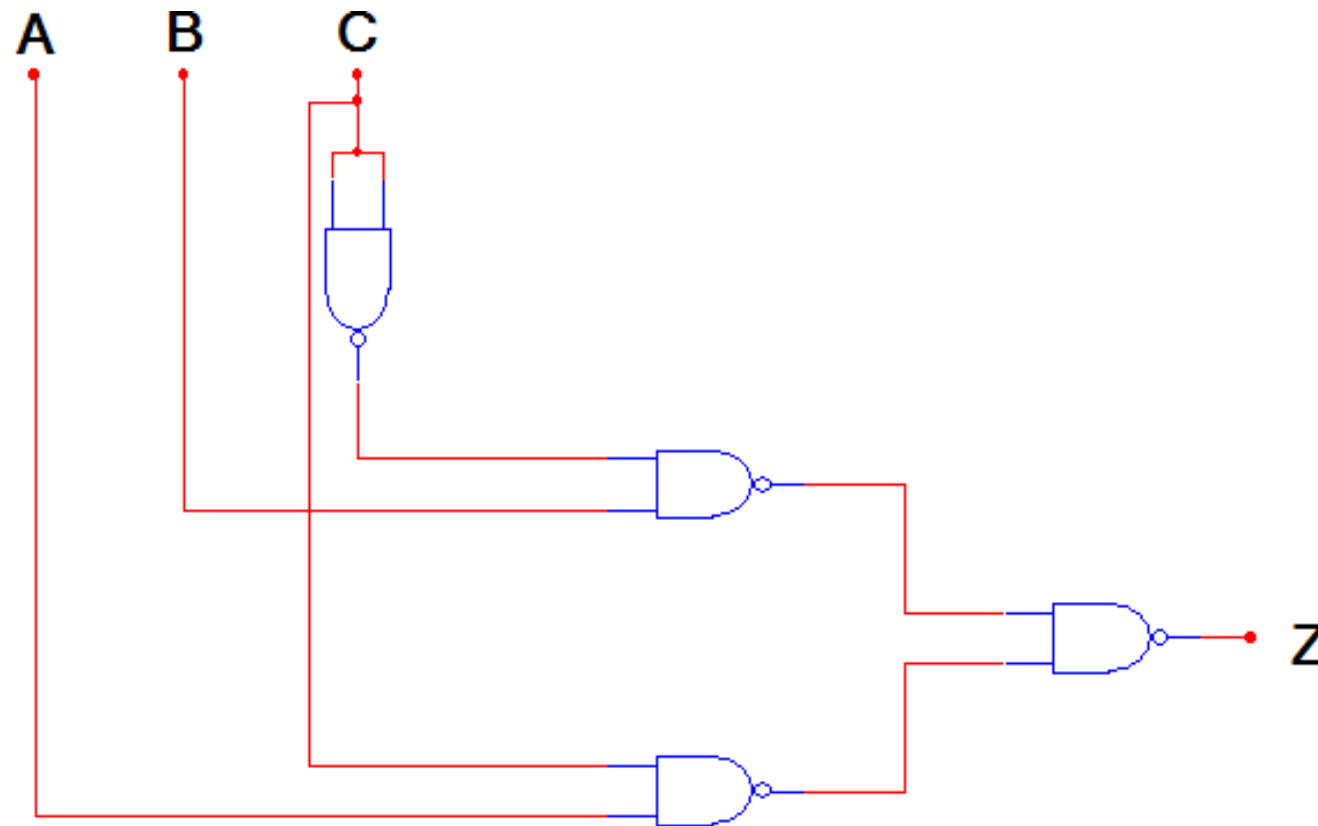


Step 4: Identify and eliminate any double inversions.



NAND Implementation

Step 5: Redraw the circuit.



NAND Implementation

- Simplify Boolean function and implement with nand gate
- $Y(X,Y,Z) = \sum m(1,3,7)$

Sum		YZ	Y'Z'	Y'Z	YZ	YZ'
X		00	01	11	10	
x'	0	0	1	1	0	
x	1	0	0	1	0	

$$Y = X'Z + YZ$$

Self Task

- Implement the this Boolean express using only NAND gates

NOR Implementation

Class work

Consensus Theorem

Redundancy theorem is used as a Boolean algebra trick in Digital Electronics. It is also known as the Consensus Theorem:

Conditions for applying Redundancy theorem are:

1. Three variables must present in the expression. Here A, B and C are used as variables.
2. Each variables is repeated twice.
3. One variable must present in complemented form.

After applying this theorem we can only take those terms which contains the complemented variable

Consensus Theorem

In SOP form

$$AB + B'C + AC = AB + B'C$$

In POS form

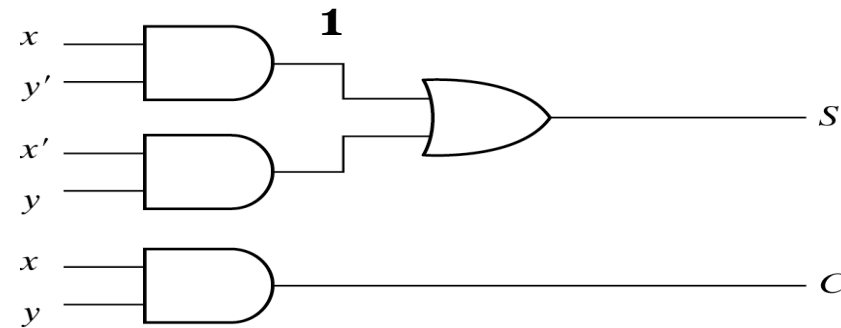
$$(A+B)(B'+C)(A+C) = (A+B)(B'+C)$$

Fault in Logic Circuit

Stack at 0 and stack at 1 gate

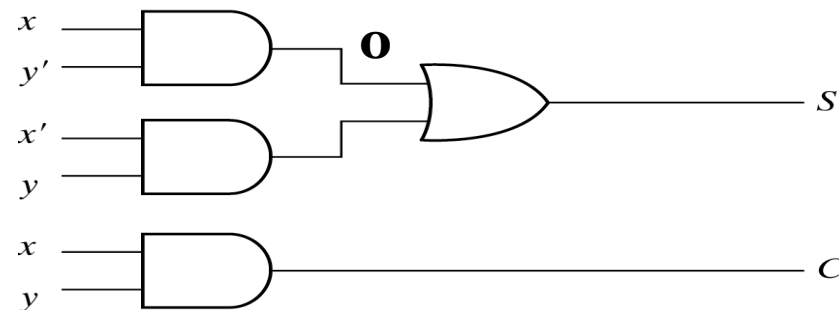
Stack at 1

The output of a given point will stay 1 no matter what is the circuit connection



Stack at 0

The output of a given point will stay 0 no matter what is the circuit connection



Minimum Two input Nand gate for multi input Nand gate

Two input NAND to implement n input AND : $2(n-1)$

Ex: how many two input NAND gate required to implement 3 input AND gate ? Ans:4

Two input NAND to implement n input NAND: $(2n-3)$

how many two input NAND gate required to implement 3 Input NAND gate ? Ans:3

Self-duality

A function is said to be Self dual if and only if its dual is equivalent to the given function, i.e., if a given function is $f(X, Y, Z) = (XY + YZ + ZX)$ then its dual is, $fd(X, Y, Z) = (X + Y).(Y + Z).(Z + X)$ (fd = dual of the given function) $= (XY + YZ + ZX)$, it is equivalent to the given function. So function is self dual.

number of Self-dual functions possible for a given function.

Let, a function has n variables then,

Number of pairs possible = $2^n/2 = 2^{(n-1)}$

the number of Self-dual functions is possible with n variables $= 2^{2^{(n-1)}}$

What is the total number of self-duals of a function which has 3 variables X, Y , and Z ?

$$= 2^{2^{(3-1)}}$$

$$= 2^{2^2}$$

$$= 2^4$$

$$= 16$$